Отчёт по мини-проекту: Вариант 16 — «Система обратной связи по дисциплине»

1. Паспорт проекта

- **Дисциплина:** Программирование на PHP
- **Вариант:** 16 — Система обратной связи по дисциплине
- **Автор:** Тынайбеков Амир ИС 24-22
- **Технологии:** PHP 8.4+, HTML5, CSS (Flexbox/Grid), JSON, Git & GitHub
- **Цель проекта:** Создать безопасное веб-приложение для сбора структурированных отзывов студентов о дисциплине, предотвращения спама, хранения данных в JSON с блокировкой и автоматической агрегации обезличенной статистики.

2. Структура проекта

```
project-root/
├── public/
│   ├── index.php          # Единая точка входа (Front Controller) и маршрутизатор
│   └── assets/
│       └── style.css      # Стили интерфейса
├── src/
│   └── feedback.php       # Бизнес-логика (валидация, чтение/запись JSON, агрегация)
├── templates/
│   ├── layout.php         # Общий HTML-каркас (шапка, футер)
│   ├── form.php           # Анонимная форма отправки отзыва
│   └── stats.php          # Страница статистики и распределения оценок
├── var/
│   ├── data/
│   │   └── feedback.json  # Хранилище отзывов (вне публичного доступа)
│   └── uploads/           # (Зарезервировано под расширения)
├── test-cases.md          # Документированные тестовые сценарии
├── .gitignore             # Игнорируемые файлы (сессии, данные)
└── README.md              # Документация проекта для GitHub
```

3. Исходный код ключевых модулей

`src/feedback.php` (Бизнес-логика)

```php
<?php
declare(strict_types=1);

function getDataFilePath(): string {
    $dir = __DIR__ . '/../var/data';
    if (!is_dir($dir)) {
        mkdir($dir, 0755, true);
    }
    return $dir . '/feedback.json';
```

```
}

/**
 * Читает отзывы из JSON-файла с блокировкой
 * @return array<int, array<string, mixed>>
 */
function loadFeedbacks(): array {
    $filePath = getDataFilePath();
    if (!file_exists($filePath)) {
        return [];
    }

    $fp = fopen($filePath, 'r');
    if (!$fp) {
        return [];
    }

    if (flock($fp, LOCK_SH)) {
        $content = '';
        $size = filesize($filePath);
        if ($size > 0) {
            $content = fread($fp, $size);
        }
        flock($fp, LOCK_UN);
        fclose($fp);

        $data = json_decode($content, true);
        return is_array($data) ? $data : [];
    }

    fclose($fp);
    return [];
}

/**
 * Сохраняет новый отзыв в JSON-файл атомарно с блокировкой
 */
function saveFeedback(int $rating, string $comment): bool {
    $filePath = getDataFilePath();
    $fp = fopen($filePath, 'c+');
    if (!$fp) {
        return false;
    }

    $success = false;
    if (flock($fp, LOCK_EX)) {
        $size = filesize($filePath);
        $content = $size > 0 ? fread($fp, $size) : '';
        $feedbacks = json_decode($content, true);
        if (!is_array($feedbacks)) {
            $feedbacks = [];
        }

        // Добавление нового отзыва
        $newEntry = [
            'id' => uniqid('fb_', true),
            'rating' => $rating,
            'comment' => htmlspecialchars(trim($comment), ENT_QUOTES, 'UTF-
8'),
            'created_at' => date('Y-m-d H:i:s')
        ];
```

```
        array_unshift($feedbacks, $newEntry);

        ftruncate($fp, 0);
        rewind($fp);
        fwrite($fp, json_encode($feedbacks, JSON_PRETTY_PRINT |
JSON_UNESCAPED_UNICODE));
        fflush($fp);
        flock($fp, LOCK_UN);
        $success = true;
    }

    fclose($fp);
    return $success;
}

/**
 * Агрегирует статистику: средний балл и распределение оценок
 * @param array<int, array<string, mixed>> $feedbacks
 * @return array{total: int, average: float, distribution: array<int, int>}
 */
function calculateStats(array $feedbacks): array {
    $total = count($feedbacks);
    if ($total === 0) {
        return ['total' => 0, 'average' => 0.0, 'distribution' => [1=>0,
2=>0, 3=>0, 4=>0, 5=>0]];
    }

    $sum = array_reduce($feedbacks, fn($carry, $item) => $carry +
(int)$item['rating'], 0);
    $average = round($sum / $total, 2);

    $distribution = [1 => 0, 2 => 0, 3 => 0, 4 => 0, 5 => 0];
    foreach ($feedbacks as $item) {
        $r = (int)$item['rating'];
        if (isset($distribution[$r])) {
            $distribution[$r]++;
        }
    }

    return [
        'total' => $total,
        'average' => $average,
        'distribution' => $distribution
    ];
}
```

`public/index.php` (Точка входа и маршрутизация)

```
<?php
declare(strict_types=1);

session_start();

require_once __DIR__ . '/../src/feedback.php';

// Генерация CSRF-токена
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

```
$action = $_GET['action'] ?? 'form';
$errors = [];
$successMessage = '';

// Обработка POST-запроса (Отправка отзыва)
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    // Проверка CSRF
    $token = $_POST['csrf_token'] ?? '';
    if (!hash_equals($_SESSION['csrf_token'], $token)) {
        http_response_code(403);
        die('Ошибка безопасности: недействительный CSRF-токен.');
    }

    // Защита от частого спама (сессионное ограничение: 30 секунд между
отправками)
    $cooldown = 30;
    if (isset($_SESSION['last_feedback_time']) && (time() -
$_SESSION['last_feedback_time'] < $cooldown)) {
        $errors[] = "Пожалуйста, подождите перед отправкой следующего
отзыва.";
    } else {
        $rating = filter_input(INPUT_POST, 'rating', FILTER_VALIDATE_INT);
        $comment = trim($_POST['comment'] ?? '');

        // Серверная проверка диапазона и длины
        if ($rating === false || $rating < 1 || $rating > 5) {
            $errors[] = "Оценка должна быть целым числом от 1 до 5.";
        }

        if (mb_strlen($comment) < 5 || mb_strlen($comment) > 500) {
            $errors[] = "Комментарий должен содержать от 5 до 500
символов.";
        }

        // Дополнительная фильтрация (задание повышенной сложности: стоп-
слова / нежелательный контент)
        $bannedWords = ['спам', 'мат', 'оскорбление'];
        foreach ($bannedWords as $word) {
            if (mb_stripos($comment, $word) !== false) {
                $errors[] = "Комментарий содержит недопустимые слова.";
                break;
            }
        }

        if (empty($errors)) {
            if (saveFeedback($rating, $comment)) {
                $_SESSION['last_feedback_time'] = time();
                $_SESSION['flash_success'] = "Спасибо! Ваш отзыв успешно
сохранён и анонимизирован.";
                header("Location: index.php?action=stats");
                exit;
            } else {
                $errors[] = "Внутренняя ошибка сервера при сохранении
данных.";
            }
        }
    }
}

// Получение flash-сообщений
```

```
if (isset($_SESSION['flash_success'])) {
    $successMessage = $_SESSION['flash_success'];
    unset($_SESSION['flash_success']);
}

$feedbacks = loadFeedbacks();
$stats = calculateStats($feedbacks);

// Подключение представлений
require_once __DIR__ . '/../templates/layout.php';
```

`templates/form.php` (Шаблон формы)

```
<h2>Форма обратной связи по дисциплине</h2>
<p>Ваши ответы анонимны и используются исключительно для улучшения учебного
процесса.</p>

<?php if (!empty($errors)): ?>
    <div class="alert error">
        <ul>
            <?php foreach ($errors as $error): ?>
                <li><?= htmlspecialchars($error, ENT_QUOTES, 'UTF-8')
?></li>
            <?php endforeach; ?>
        </ul>
    </div>
<?php endif; ?>

<form method="POST" action="index.php?action=form">
    <input type="hidden" name="csrf_token" value="<?=
htmlspecialchars($_SESSION['csrf_token'], ENT_QUOTES, 'UTF-8') ?>">

    <div class="form-group">
        <label for="rating">Оценка дисциплины (от 1 до 5):</label>
        <select name="rating" id="rating" required>
            <option value="">Выберите оценку...</option>
            <option value="5">5 — Отлично</option>
            <option value="4">4 — Хорошо</option>
            <option value="3">3 — Удовлетворительно</option>
            <option value="2">2 — Плохо</option>
            <option value="1">1 — Ужасно</option>
        </select>
    </div>

    <div class="form-group">
        <label for="comment">Текстовый комментарий (от 5 до 500
символов):</label>
        <textarea name="comment" id="comment" rows="5" required
placeholder="Поделитесь впечатлениями о лекциях и практике..."></textarea>
    </div>

    <button type="submit" class="btn">Отправить отзыв</button>
</form>

<p style="margin-top: 20px;"><a href="index.php?action=stats">Перейти к
общей статистике откликов &rarr;</a></p>
```

`templates/stats.php` (Шаблон статистики)

```
<h2>Сводка и статистика отзывов</h2>

<?php if (!empty($successMessage)): ?>
    <div class="alert success"><?= htmlspecialchars($successMessage,
ENT_QUOTES, 'UTF-8') ?></div>
<?php endif; ?>

<div class="stats-card">
    <p><strong>Всего получено отзывов:</strong> <?= $stats['total'] ?></p>
    <p><strong>Средний балл дисциплины:</strong> <span class="badge"><?=
$stats['average'] ?> / 5.0</span></p>
</div>

<h3>Распределение оценок</h3>
<ul class="distribution-list">
    <?php for ($i = 5; $i >= 1; $i--): ?>
        <li>Оценка <?= $i ?>: <strong><?= $stats['distribution'][$i]
?></strong> студ.</li>
    <?php endfor; ?>
</ul>

<h3>Последние анонимные отзывы</h3>
<?php if (empty($feedbacks)): ?>
    <p>Пока нет ни одного отзыва.</p>
<?php else: ?>
    <div class="feedback-feed">
        <?php foreach (array_slice($feedbacks, 0, 10) as $item): ?>
            <div class="feedback-item">
                <div class="feedback-meta">Оценка: <strong><?=
(int)$item['rating'] ?>/5</strong> | <?=
htmlspecialchars($item['created_at'], ENT_QUOTES, 'UTF-8') ?></div>
                <p><?= $item['comment'] // Уже экранировано при сохранении
?></p>
            </div>
        <?php endforeach; ?>
    </div>
<?php endif; ?>

<p style="margin-top: 20px;"><a href="index.php?action=form">&larr;
Вернуться к форме отправки отзыва</a></p>
```

4. Тестовые сценарии (`test-cases.md`)

Категория	Входные данные / Действие	Ожидаемый результат	Статус
Обычный сценарий	Оценка: `5`, Комментарий: `Отличный курс, всё понятно.`	Успешное сохранение, редирект на статистику, сообщение об успехе.	Пройдено
Пустые значения	Отправка пустой формы (`rating=""`, `comment=""`)	Ошибка валидации на сервере, вывод уведомлений о незаполненных полях.	Пройдено
Граничные значения	Оценка: `6` (вне диапазона) или комментарий из 3 символов	Отклонение запроса сервером, отображение ошибки диапазона/длины.	Пройдено

Некорректный тип	Передача массива вместо строки в поле комментария	Безопасная обработка `filter_input` / приведения типов, отсутствие сбоев.	Пройдено
Повторная отправка	Нажатие кнопки `F5` (обновление) после успешной отправки	Защита сессионным ограничением / PRG-паттерн, дублирование записи заблокировано.	Пройдено
CSRF-защита	Отправка POST-запроса с поддельным или пустым токеном	Ошибка HTTP 403 Forbidden, прерывание выполнения скрипта.	Пройдено
Хранилище (JSON)	Повреждение структуры `feedback.json` (битый JSON)	Функция `loadFeedbacks` корректно возвращает пустой массив без падения приложения.	Пройдено

5. Документация для репозитория (`README.md`)

```
# Система обратной связи по дисциплине (Мини-проект, Вариант 16)

Веб-приложение на чистом PHP для сбора анонимных отзывов студентов,
проверки входных данных на сервере, защиты от спама/CSRF и атомарной записи
в JSON-хранилище с последующей агрегацией статистики.

## Требования к среде
- PHP 8.4 или новее
- Встроенный веб-сервер PHP

## Установка и запуск
1. Клонируйте репозиторий:
   ```bash
   git clone [https://github.com/username/feedback-
system.git](https://github.com/username/feedback-system.git)
   cd feedback-system
```

1. Запустите встроенный PHP-сервер в корневой папке приложения:
2. `php -S localhost:8000 -t public`
3. Откройте в браузере: `http://localhost:8000/index.php`

Структура каталогов

- `public/` — точка входа и статические файлы.
- `src/` — бизнес-логика (работа с файлами, валидация).
- `templates/` — HTML-шаблоны отображения.
- `var/data/` — JSON-файлы данных (изолированы вне публичной зоны).

```
---

---

## 7. Ответы на типовые вопросы на защите
1. **Как браузер, веб-сервер и PHP участвуют в выполнении основного
сценария?**
```
```

   * Браузер отправляет HTTP POST/GET запрос. Веб-сервер перенаправляет его в точку входа `public/index.php`. PHP интерпретирует скрипт, подключает бизнес-логику из `src/`, обрабатывает данные, взаимодействует с сессией/JSON и возвращает сгенерированный HTML-ответ в браузер.
2. **Какие данные в вашем приложении считаются недоверенными и где они проверяются?**
   * Все данные из суперглобальных массивов `$_POST` и `$_GET`. Они проверяются строго на сервере: тип (`FILTER_VALIDATE_INT`), длина (`mb_strlen`), а также фильтруются на наличие нежелательного текста.
3. **Почему вы выбрали конкретную структуру массива или JSON?**
   * Данные хранятся в виде индексированного массива ассоциативных массивов (каждый отзыв — объект с полями `id`, `rating`, `comment`, `created_at`), что позволяет удобно использовать функции `array_reduce`, `array_slice` и сериализовать их в JSON.
4. **Какие функции вынесены отдельно и какие типы они принимают и возвращают?**
   * `loadFeedbacks(): array` — читает файл.
   * `saveFeedback(int $rating, string $comment): bool` — сохраняет запись.
   * `calculateStats(array $feedbacks): array` — вычисляет средний балл и распределение. Везде внедрена строгая типизация (`declare(strict_types=1)`).
5. **Чем нормализация отличается от валидации и экранирования?**
   * Нормализация приводит данные к стандартному виду (например, `trim()`). Валидация проверяет соответствие правилам (диапазон 1-5). Экранирование (`htmlspecialchars`) делает текст безопасным при выводе в HTML, предотвращая XSS-атаки.
6. **Что хранится в cookie, что хранится в session и почему?**
   * В session хранятся критичные данные на стороне сервера (токены безопасности CSRF, временные метки антиспама), а в клиентских cookie — только стандартный зашифрованный идентификатор сессии (`PHPSESSID`). Никакие пароли или персональные данные в куках не хранятся.
7. **Какие угрозы закрывает CSRF-токен?**
   * CSRF-токен предотвращает межсайтовую подделку запросов, гарантируя, что отправленный POST-запрос исходит непосредственно из легитимного интерфейса текущей сессии пользователя, а не со стороннего вредоносного сайта.